

第十讲 操作系统安全

Part I 操作系统安全的一般性原理

操作系统安全是等级保护的关键技术

一、操作系统安全问题的来源

1、操作系统面临的安全威胁

黑客攻击 蠕虫 拒绝服务攻击 木马程序 后门 隐蔽通道 信息篡改 机密信息泄露 计算机病毒 逻辑炸弹 非法访问 破解口令 (password cracking) 开天窗 (trapdoor) 走后门 (backdoor) 安放特洛伊木马 (Trojan horse) …

2、关于漏洞的考量

Debian Linux 第一，Android 第二。

3、关于攻击的考量（以隐通道攻击为例）

1973 年，B. W. Lampson 通过对程序的限制问题（Confinement Problem）的研究提出了隐通道的概念。隐通道是一类不易被发现的数据泄密途径。

程序的限制问题涉及程序的调用与信息的传送

- 设程序 B 是由程序 A 调用运行的。
- 所谓对程序 B 的限制是指：制止程序 B 在运行期间向其他程序（程序 A 除外）传送信息。
- 程序 B 向它的 owner 传送有关程序 A 的信息属于信息泄漏。

(1) Lampson 关于隐通道的定义

按常规不会用于传送信息但却被利用于泄漏信息的信息传送渠道。

例如：一个程序通过对系统负载的影响的变化情况来向另一个程序暗示某种信息。

(2) TCSEC 关于隐通道的定义

可以被进程利用来以违反系统安全策略的方式进行非法传输信息的通信通道

① 存储隐通道

举例：

❖ 限制：进程 P 不能与进程 Q 通信。

❖ 共享：文件系统。

❖ 目的：进程 P 想发送一条消息给进程 Q。

❖ 一种使用存储隐通道的方法：

➤ 进程 P 在两进程都能读的一个目录中创建一个名为“send”的文件

➤ 进程 Q 要读消息前，Q 删除文件 send

➤ 进程 P 创建一个文件，命名为 0bit 或者 1bit 来代表要传输的比特；

➤ 进程 Q 记录这个比特，并删除这个文件

➤ ...

➤ 最后进程 P 创建一个名为“end”的文件。

通信结束。

②时间隐通道

举例：

❖ Marvin, S., et al., Program confinement in KVM/370, in

Proceedings of the 1977 annual conference. 1977, ACM.

❖ 针对 KVM/370 系统的研究发现，可以基于两个虚拟机接收的 CPU 时间片来建立隐通道

- 若发送方想要发送比特 0，则立即释放 CPU
- 否则它将使用全部的时间片
- 接收方通过获得 CPU 的速度可以推断出前者发送的是 0 还是 1

❖ 共享资源： CPU 时间

4、关于实际安全需求的考量

(1) 物联网中操作系统的安全性问题日趋突出

对物联网的具体利用进行分析，发现其核心 —— 也就是操作系统安全问题表现得越来越突出，严重影响了物联网的利用效果，并对物联网的实际价值发挥造成了影响，所以解决操作安全成为了物联网应用中亟待解决的难题。

①直接沿用原有的安全机制。例如， Android Things 直接沿用了 Android 系统的一些基础安全机制，并没有深入分析物联网设备实际的软硬件特性与需求

②缺乏对终端系统安全设计。现有的物联网操作设计时普遍只关注其功能要求

③没有充分利用设备自身硬件架构安全特性

(2) 虚拟化系统中操作系统的安全性问题

伴随着大数据和云时代的到来，虚拟化技术的应用已经越来越成熟。如何运用虚拟化技术给终端客户提供一个安全、稳定、高效的桌面环境，成为了一个日益热门的话题。

由著名波兰黑客 Joanna Rutkowska 发布的基于 Xen 和 Linux 的开源操作系统 — Qubes 充分利用了虚拟化技术（基于安全虚拟机 Xen），所有用户应用程序都运行在 AppVM（基于 Linux 的轻量级虚拟机）中，彼此隔离。而联网代码使用 IOMMU/VT-d 放在一个非特权虚拟机中，在特权域（dom0）中没有任何联网代码。许多系统级组件放在沙盒中，以避免互相影响。

(3) 嵌入式系统中操作系统的安全新问题

适航审定是大飞机研制中的关键环节，试飞测试是适航审定任务实施的重要基础；纵观国内外大飞机飞行试验均非常重视机载数据实时处理；在 C919 型号中对机载数据实时处理系统进行了研究，以 VxWorks 嵌入式实时操作系统为平台，通过对现有机载实时处理系统功能模块进行整合以及软件优化设计来提高 C919 机载数据实时处理系统性能。

嵌入式操作系统是嵌入式系统的核心，也是信息系统安全的重要保障，如果缺乏这个安全的根基，构建在操作系统上的应用系统以及整个嵌入式系统的安全性将得不到根本保障，因此提高嵌入式操作性系统的可靠性、安全性非常有必要。**传统的嵌入式操作系统**

安全技术将操作系统所有安全相关功能集中在安全核心，但安全核心的开发使得操作系统内核变得更加庞大，导致无法对内核通过较高安全等级的安全评估和认证，也无法确保加密处理应用的真实安全性。

二、操作系统的安全技术

1、操作系统的安全机制

(1) 隔离控制

①物理隔离

例如，把不同的打印机分配给不同安全级别的用户。

②时间隔离

例如，以不同安全级别的程序在不同的时间使用计算机。

③加密隔离

例如，把文件、数据加密，使无关人员无法阅读。

④逻辑隔离

例如，把各个进程的运行限制在一定的空间，使得相互之间不感到其他进程或程序的存在，如 sandbox。

首先要对存储单元的 地址进行保护

使非法用户不能访问
那些受到保护的存储
单元

其次要对被保护的存储单元提供 各种类型的保护

最基本的保护类型是“读/写”和“只读”。

复杂一些的保护类型还包括“只执行”、“不能存取”等操作。不能存取的存储单元，若被用户存取时，系统要及时发出警报或中断程序执行。

操作系统对内存的保护主要采用逻辑隔离方法。

操作系统对存储器的保护：

I 界地址寄存器保护法

- 在 CPU 中设置一对下限寄存器和上限寄存器，存放正在执行的程序在主存中的下限和上限地址。
- 可将一个寄存器作为基址寄存器，另一寄存器作为限长寄存器（指示存储区长度）
- 每当 CPU 要访问主存，硬件自动将被访问的主存地址与界限寄存器的内容进行比较，以判断是否越界。如果未越界，则按此地址访问主存，否则将产生程序中断 — 越界中断（存储保护中断）。

II 内存标志法

III 锁保护法

“锁”是指为存储区设置的特定数字。

使用锁保护方式具有以下优点：

- 因为不同的存储区可以分配相同的锁，因此用户可以用同一钥匙打开不连续的若干区域。
- 只要锁和钥匙定义不同的对应方案，就可以使钥匙具有不同的优先级。

IV 特征位保护法

特征位即在每一个存储单元的前面设置一组特征位。

特征位	数据指令代码
-----	--------

(2) 访问控制

访问控制是操作系统的安全控制核心。访问控制是确定谁能访问系统，能访问系统何种资源以及在何种程度上使用这些资源。访问控制包括对系统各种资源的存取控制。

访问控制的三项基本任务：身份认证、确定访问权限和实施访问控制的权限。

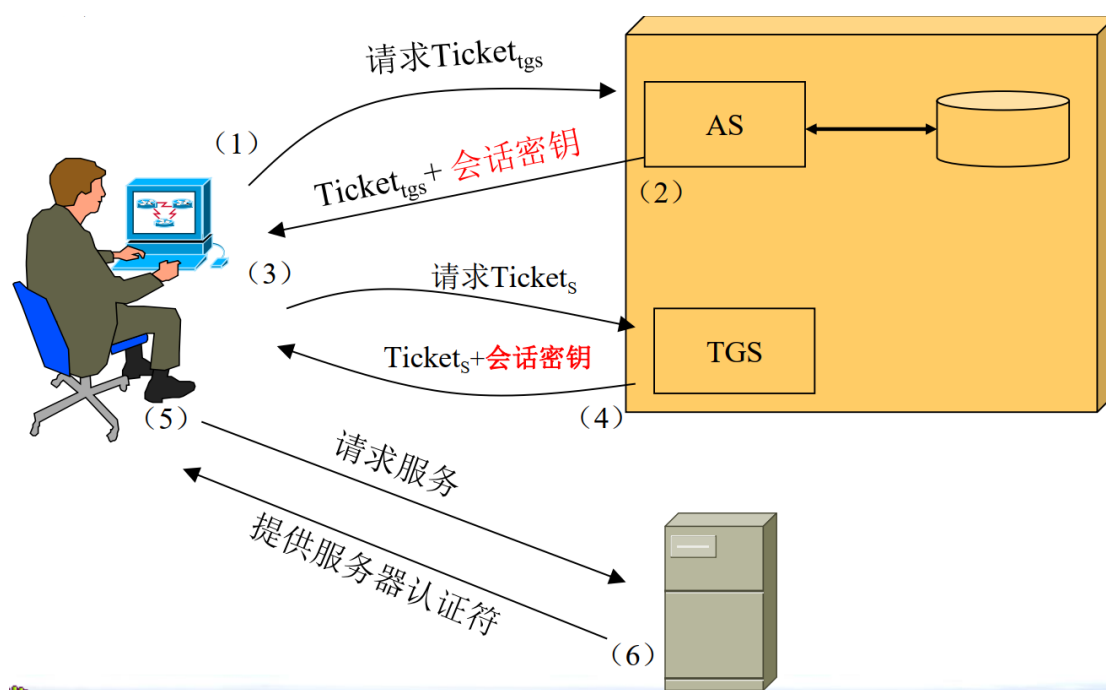
①一次性口令认证 (OTP)

OTP 的主要思路是：在登录过程中加入不确定因素，使每次登录过程中传送的口令信息都不相同，以提高登录过程安全性。

例如：登录密码=MD5(用户名+密码+时间)。系统接收到登录口令后做一个验算即可验证用户的合法性。

不是解决安全问题的终极方案。银行的 U 盾为例，随机种子。

②Kerberos 认证



会话密钥是对称密钥。

两个票据，三台服务器，四个参与者，五个会话密钥。

Kerberos 报文交换第一轮：获取票据许可票据

Step (1) 用户请求票据许可票据

- 用户请求票据许可票据的工作在登录工作站时进行。
- 登录时用户被要求输入用户名，输入后系统会向认证服务器

AS 以明文方式发送一条包含用户和 TGS 服务两者名字的请求。

$$C \rightarrow AS : ID_C \parallel ID_{tgs} \parallel TS_1$$

Step (2) AS 发放票据许可票据和会话密钥

认证服务器检查用户如果有效，随机产生一个用户和 TGS 之间的会话密钥 $K_{c,tgs}$ ，然后创建一个票据许可票据 $Ticket_{tgs}$ （票据许可票据使用 K_{tgs} 加密）。认证服务器向用户发送票据许可票据和会话密钥 $K_{c,tgs}$ ，发送的消息用只有用户和 AS 知道的 K_c 来加密（ K_c 的值基于用户的密码）。

$$AS \rightarrow C : E_{K_c}[K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}]$$

$$Ticket_{tgs} = E_{K_{tgs}}[K_{c,tgs} \parallel ID_C \parallel AD_C \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2]$$

Kerberos 报文交换第二轮：获取访问票据

Step (3) C 请求服务器票据

用户工作站收到 AS 回应后，要求用户输入密码，将密码转化为 DES 密钥 K_c ，然后将 AS 发回的信息解开，将 $Ticket_{tgs}$ 和 $K_{c,tgs}$ 保存用于以后的通信，为了安全性用户密码和密钥 K_c 被删掉。

一个票据只能申请一个特定的服务，所以用户必须为每一个服务器申请新的票据，用户可以从 TGS 处得到票据 $Ticket_S$ 。

用户首先向 TGS 发出申请服务器票据的请求。请求信息中包含 TGS 服务器的名字，上一步中得到的请求 TGS 服务的加密票据 $Ticket_{tgs}$ ，还有用会话密钥 $K_{c,tgs}$ 加密过的 Authenticator 信息。

$$C \rightarrow TGS: ID_{tgs} || Ticket_{tgs} || Authenticator_C$$

$$Authenticator_C = E_{K_{c,tgs}}[ID_C || AD_C || TS_3]$$

$$Ticket_{tgs} = E_{K_{tgs}}[K_{c,tgs} || ID_C || AD_C || ID_{tgs} || TS_2 || Lifetime_2]$$

Step (4) TGS 发放服务器票据和会话密钥

TGS 收到请求后，用密钥 K_{tgs} 和会话密钥 $K_{c,tgs}$ 分别解开 $Ticket_{tgs}$ 和 $Authenticator_C$ ，根据两者的信息鉴定用户身份是否有效。如果有效，TGS 生成用于 C 和 S 之间通信的会话密钥 $K_{C,S}$ ，并生成 C 申请 S 服务的票据 $Ticket_s$ ，其中包含 C 和 S 的名字、C 的网络地址、当前时间、有效时间和刚才产生的会话密钥 $K_{C,S}$ 。

TGS 最后将加密后的票据 $Ticket_s$ 和会话密钥 $K_{C,S}$ 用用户和 TGS 之间的会话密钥 $K_{c,tgs}$ 加密后发送给用户。用户 C 得到回答后，用 $K_{c,tgs}$ 解密，得到所请求的票据和会话密钥。

$$TGS \rightarrow C: E_{K_{c,tgs}}[K_{C,S} || ID_S || TS_4 || Ticket_s]$$

$$Ticket_s = E_{K_S}[K_{C,S} || ID_C || AD_C || ID_S || TS_4 || Lifetime_4]$$

Kerberos 报文交换第三轮：**获取服务**

Step (5) 工作站将票据和认证符发给服务器

$$C \rightarrow S: Ticket_s || Authenticator_C$$

$$Ticket_s = E_{K_S}[K_{C,S} || ID_C || AD_C || ID_S || TS_4 || Lifetime_4]$$

$$Authenticator_C = E_{K_{C,S}}[ID_C || AD_C || TS_5]$$

Step (6) 服务器验证票据 TicketS 和认证符中的匹配，然后许可访问服务。如果需要双向认证，服务器返回一个认证符

$$S \rightarrow C : E_{K_{C,S}} [TS_5 + 1]$$

思考题：对于物联网这样一个完全分布式架构的网络，其身份认证面临哪些困难和挑战？

在物联网环境中部署身份验证和访问控制机制时，有许多因素会使任务复杂化。这是因为大多数设备的处理能力、存储空间、带宽和能量都是有限的。由于常见身份验证协议的通信开销，大多数传统身份验证和授权技术过于复杂，无法在资源受限的物联网设备上运行。另一个问题是，设备有时部署在可能无法或无法提供物理安全的区域。还有非常广泛的硬件和软件堆栈需要考虑。这导致大量设备通过多种标准和协议进行通信——这与更传统的计算环境不同。例如，研究人员确定了物联网环境中至少 84 种不同的身份验证机制 这些机制要么是在 2019 年提出的，要么是在 2019 年投入生产的。由于缺乏标准和物联网特定的访问控制模型，保护设备和网络安全任务变得更加复杂。

改进物联网访问控制的方法

任何试图管理数千个分布广泛的物联网设备的集中访问管理模式都有其局限性，没有一种方法适用于所有场景。寻求开发去中心化物联网访问控制服务的供应商正在研究区块链技术如何消除集中式系统造成的问题。网络管理员和安全团队必须紧跟最新发展，因为它们可能在不久的将来带来真正可扩展的服务产品。

在此之前，每一个物联网设备必须有一个唯一的身份，当设备试图连接到网关或中央网络时，可以对其进行身份验证。有些设备仅根据其 IP 或 MAC（媒体访问控制）地址进行标识，而其他设备则可能安装了证书。但是识别任何类型设备的更好方法是通过机器学习。除了行为分析（如 API、服务和数据库请求）之外，还可以使用静态特性来完成此任务，以更好地确保设备的身份。身份和身份验证行为的结合还提供了根据上下文不断调整访问控制决策的能力——即使对于资源有限的设备也是如此。

这种基于属性的访问控制模型根据对设备、资源、操作和上下文进行分类的一系列属性来评估访问请求。它还提供更多动态访问控制功能。可以基于上下文属性中的更改实时更新对操作和请求的批准。但是，它确实需要管理员选择和定义一组属性和变量，以构建一套全面的访问控制规则和策略。

操作系统的安全模型

I 访问监控模型

单级模型，最简单的安全模型，是体现有限型访问控制的模型。它对每一个主体-客体对，只作“可”还是“否”的访问判定。这种模型论证比较脆弱，它所表达的安全需求没有特别详尽的说明。

II “格”模型

多级模型。把用户（主体）和信息（客体）进一步按密级和类别划分。访问控制根据“工作需要，给予最低权限”的原则，只有

当主体的密级等于或高于客体，主体的类别等于或包含客体时，主体才能访问客体。

III Bell-La Padula 模型

多级模型，用于防止信息泄漏，而不是防止主体对客体的非授权修改。它们只处理了信息的保密问题，而没有解决信息的完整性问题。

IV BIBA 模型

多级模型，保证信息流完整性的控制模型，它把主体和客体按类似密级那样的完整级进行分类。完整级是一个由低到高的序列，其访问遵循“简单完整性”和“完整性”两条原则。

安全操作系统的设计原则

- 最小权限原则。赋予主体完成工作所必需的最小权限，减少不必要的损害。
- 完整性原则。对所有保护对象的每次访问都必须接受检查。
- 经济性原则。采用最简单、最直接的手段，对系统进行全面、充分的检查和验证：另外，安全措施代价应该小于被保护对象的价值。
- 公开性原则。系统的安全性不能建立在对系统某些技术保密的基础上，而应让这些技术经受充分的检验。
- 权限分离原则。理想的访问权应取决于两个以上条件的同时满足。

- 最小共同性原则。用物理或逻辑上的隔离来减少共享可能造成的危害。

- 易用性原则。系统人机接口的安全措施必须简单易用。

- 默认安全原则。未经说明或权限不明的访问作为不允许访问处理。

所谓“安全操作系统设计”，就是指安全性必须在操作系统的各个部分予以考虑，安全性必须在操作系统开发的初期就予以考虑。

(3) 信息加密

①加密机制用于安全传输

Internet 协议安全性 (IPSec) 是一种开放标准的框架结构，通过使用加密的安全服务以确保在 IP 网络上进行保密而安全的通信。

作为网络操作系统的 Windows 2003 Server，在分析它的安全机制时，也应该考虑到 IP 安全策略机制。

一个 IPSee 安全策略由 IP 筛选器和筛选器操作两部分构成。其中 IP 筛选器决定哪些报文应当引起 IPSee 安全策略的关注，筛选器

操作是指“允许”还是“拒绝”报文的通过。在 Windows 2003Server 系统的服务端和客户端都提供了对 IPSee 的支持，从而增强了安全性。

②加密机制用于文件安全

- 随机生成一个文件加密密钥 (FEK)，用来加密和解密文件

- FEK 会被当前帐户的公钥进行加密，加密后的 FEK 副本保存在文件\$EFS 属性的 DDF 字段

- 要想解密文件，首先必须用当前用户的私钥去解密 FEK，然后用 FEK 去解密文件

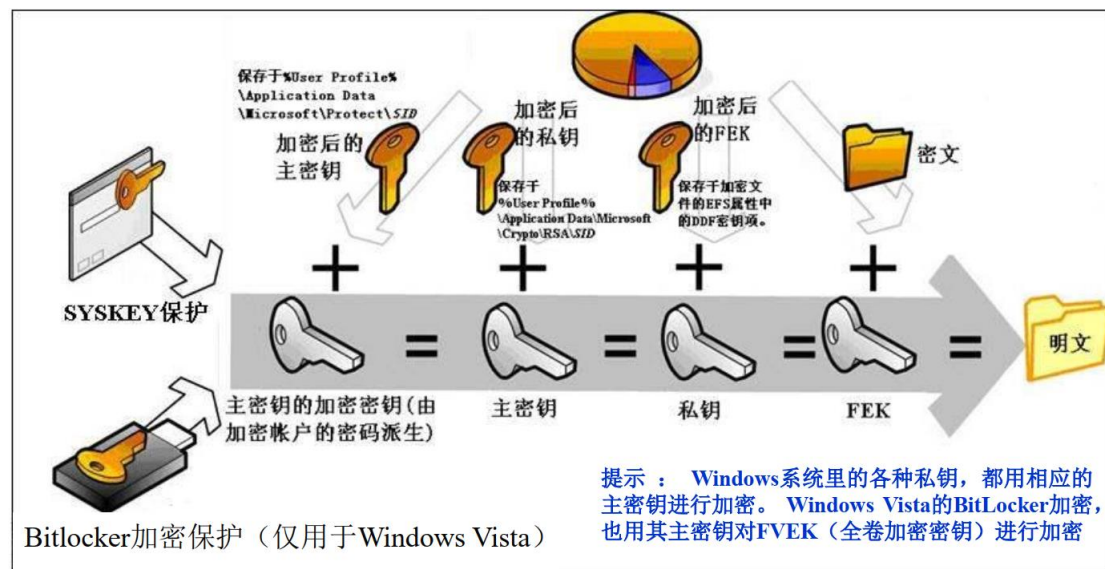
EFS 加密实际上综合了对称加密和不对称加密

加密操作后，Windows 系统状态栏会自动提示用户进行备份加密密钥，点击后会出现“备份文件加密证书和密钥”的对话框，选择“现在备份”，会出现证书导出向导。

③Microsoft 的 BitLocker 加密磁盘

Windows 7 企业版及以上版本或旗舰版保护整个磁盘不被脱机攻击，也不会被 Windows 外的其他任何系统所访问。

④EFS 和 Bitlocker 的结合



Windows 系统还会对 EFS 添加两层保护措施：

(1) Windows 会用 64 字节的主密钥（Master Key）对私钥进行加密，加密后的私钥保存在以下文件夹：`%UserProfile%/Application Data/Microsoft/Crypto/RSA/SID`

(2) 为了保护主密钥，系统会对主密钥本身进行加密（使用的密钥由帐户密码派生而来），加密后的主密钥保存在以下文件

夹： %UserProfile%/Application Data/Microsoft/Protect/SID

(4) 审计跟踪

要求任何影响系统安全性的行为都被跟踪和记录在案，安全系统拥有把用户标识与它被跟踪和记录的行为联系起来的能力。

- 审计信息必须有选择性的保留和保护，所有与安全相关的事件记录在审计日志文件中，所有审计数据必须防止受到未授权的访问、修改和破坏，以作为日后对事件调查的依据。

- 审计系统能记录以下事件：和标识与鉴别机制相关的事件、将客体导入用户地址空间的操作、删除客体、系统管理员执行的操作及其他与安全相关的事件。